

MÓDULO 2

Agente Conversacional con Memoria y Herramientas

Manuelita S.A. · OSINT & AI Project

Universidad Autónoma de Occidente

Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Mayo 2026

Campo	Valor
Proyecto	Sistema OSINT + IA — Manuelita S.A.
Módulo	Módulo 2 — Agente Conversacional con Memoria
Framework	LangChain 0.3 · ChromaDB · Streamlit
Proveedores LLM	Google Gemini API · Ollama local · HuggingFace local
Bloques completados	Bloque 1 (RAG) · Bloque 2 (Estructurado) · Bloque 3 (Router)
Fecha entrega	Mayo 2026

1. Introducción

El presente informe documenta el diseño, implementación y evaluación del **Módulo 2** del Sistema OSINT + IA para Manuelita S.A. Este módulo transforma el motor de preguntas y respuestas del Módulo 1 —basado en un único pipeline RAG— en un **agente conversacional completo** con memoria, herramientas especializadas y una interfaz de chat moderna.

La arquitectura del Módulo 2 sigue el paradigma de *agente con herramientas* propuesto por LangChain: un router decide en tiempo real qué herramienta es más adecuada para cada pregunta, combinando respuestas exactas desde datos estructurados con respuestas fundamentadas desde el corpus vectorial. La memoria conversacional permite preguntas de seguimiento contextuales, resolviendo referencias implícitas como «¿y en Perú?» o «¿cuándo fue eso?».

Objetivos del módulo

- **Bloque 1:** Indexar el corpus Markdown en ChromaDB e implementar un motor RAG con tres modos de proveedor (Gemini / Ollama / local).
- **Bloque 2:** Construir una herramienta de datos estructurados (JSON) con detección de intención por keywords, latencia 0ms y exactitud 100%.
- **Bloque 3:** Diseñar un agente router (HybridRouter) que seleccione automáticamente entre RAG y datos estructurados.
- **Bloque 4:** Integrar ConversationBufferWindowMemory para mantener el historial y resolver preguntas de seguimiento.
- **Bloque 5:** Actualizar la interfaz Streamlit a un chat conversacional con burbujas, badges de fuente y selector de proveedor.

2. Arquitectura del Sistema

El sistema está organizado en cinco bloques funcionales apilados, donde cada bloque extiende las capacidades del anterior sin romper la compatibilidad hacia atrás.

Componente	Bloque	Responsabilidad
Streamlit Chat UI	Bloque 5	Interfaz conversacional, burbujas, badges
ContextualAgent	Bloque 4	Agente con ConversationBufferWindowMemory
ManuelitaAgent (HybridRouter)	Bloque 3	Router por keywords: RAG vs Estructurado
ManuelitaStructuredTool	Bloque 2	JSON en memoria, 0ms, 100% exactitud
ManuelitaRAG (ChromaDB)	Bloque 1	Embeddings + LLM, preguntas abiertas

Lógica de routing (HybridRouter)

El router clasifica cada pregunta antes de invocar cualquier herramienta, evitando el costoso ciclo Pensamiento/Acción/Observación del agente ReAct. La decisión sigue una prioridad estricta:

Prioridad	Condición	Herramienta
1 (máxima)	Señales narrativas: «valores», «cómo», «gestiona», «preocupa»	RAG (ChromaDB)
2	Coincidencia con categoría exacta: NIT, presidente, países	Datos Estructurados
3 (default)	Pregunta sin clasificar	RAG (ChromaDB)

3. Bloque 1 — Motor RAG con ChromaDB

El motor RAG (*Retrieval-Augmented Generation*) indexa el corpus Markdown de Manuelita S.A. en ChromaDB y recupera los fragmentos más relevantes para responder preguntas abiertas. La principal innovación de este bloque es el archivo `key_facts_manuelita.md` en formato Q&A, que crea pares pregunta-respuesta con alta similitud coseno con las consultas reales.

Parámetros de indexación

Parámetro	Valor	Descripción
CHUNK_SIZE	500 chars	Tamaño de cada fragmento de texto
CHUNK_OVERLAP	80 chars	Solapamiento entre fragmentos consecutivos
DEFAULT_K	5–6 docs	Fragmentos recuperados por consulta
COLLECTION	manuelita_corpus	Nombre de la colección en ChromaDB
Chunks totales	~150	Fragmentos indexados en el corpus completo

Resultados por proveedor

Proveedor	Embeddings	LLM	Score	Tiempo/pregunta
gemini	gemini-embedding-001	gemini-2.0-flash	~95%	~3s
local	paraphrase-multilingual-MiniLM-L12-v2	llama3.2:3b (Ollama)	86%	~15s
ollama	nomic-embed-text	llama3.2:3b (Ollama)	~60%	~10s

Nota: el score con modo local (86%) es inferior al de Gemini (~95%) principalmente porque el modelo llama3.2:3b no responde bien preguntas narrativas en español. El routing es correcto en todos los casos; la calidad de la respuesta RAG mejora significativamente con Gemini.

4. Bloque 2 — Herramienta de Datos Estructurados

La herramienta de datos estructurados responde preguntas factuales exactas consultando directamente un archivo JSON en memoria, sin invocar ningún LLM ni sistema de embeddings. Esto garantiza respuestas 100% exactas con latencia prácticamente nula (~0ms).

Categorías disponibles

Categoría	Datos respondidos	Ejemplo de pregunta
nit	NIT de Manuelita S.A.	¿Cuál es el NIT?

presidente	Nombre del CEO/presidente	¿Quién dirige Manuelita?
fundacion	Año y lugar de fundación	¿En qué año fue fundada?
países	Países de operación y exportación	¿En qué países opera?
empleados	Número de colaboradores	¿Cuántos empleados tiene?
ingresos	Ingresos históricos 2019–2023	¿Cuánto facturó en 2023?
ebitda	EBITDA y margen operativo	¿Cuál es el EBITDA?
unidades	7 unidades de negocio	¿Cuáles son las subsidiarias?
carbono	Metas de sostenibilidad 2030/2040	¿Meta de carbono para 2030?
general	Resumen corporativo	Cuéntame sobre Manuelita

Métrica	Resultado
Categorías detectadas	10/10 (100%)
Score de keywords	100%
Tiempo de respuesta	~0.0s por pregunta
Dependencias externas	Ninguna (solo JSON en memoria)

5. Bloque 3 — Agente Router (HybridRouter)

El agente router es el núcleo del sistema. Recibe la pregunta del usuario y decide automáticamente qué herramienta invocar. A diferencia de un agente ReAct puro (que depende de la capacidad de razonamiento del LLM), el HybridRouter usa clasificación por keywords antes de invocar cualquier herramienta, lo que lo hace fiable incluso con modelos pequeños como llama3.2:3b.

Implementación — ManuelitaAgent

La clase ManuelitaAgent en src/langchain_app/agent.py implementa el método _route(question) con la siguiente lógica:

```
def _route(self, question):
    q = question.lower()
    # 1. Señales narrativas → siempre RAG
    if any(sig in q for sig in self._RAG_SIGNALS):
        return 'rag', 'corpus'
    # 2. Categoría estructurada detectada
    cat = self.structured._detect_category(question)
    if cat != 'general':
        return 'structured', cat
    # 3. Por defecto → RAG
    return 'rag', 'corpus'
```

Resultados — test_agente_bloque3.py

Métrica	Resultado
Routing correcto	10/10 (100%)
Estructurado ruteado	6/6
RAG ruteado	4/4
Tiempo preguntas estructuradas	~0.0s
Tiempo preguntas RAG (local)	~13s
Score keywords (local)	60% (limitado por llama3.2:3b en español)
Score keywords (gemini estimado)	~80–100%

El 100% de routing con el modo local demuestra que el HybridRouter no depende de la capacidad de razonamiento del LLM — el routing es deterministico y basado en keywords. La baja calidad de las respuestas RAG con modo local se debe exclusivamente a las limitaciones del modelo llama3.2:3b para responder en español desde contexto.

6. Bloque 4 — Memoria Conversacional

El Bloque 4 extiende el agente con ConversationBufferWindowMemory de LangChain, manteniendo los últimos N turnos de conversación. Esto permite preguntas de seguimiento como «¿y qué produce en Perú?» después de haber preguntado por los países de operación.

Componentes — memory.py

Clase / Función	Responsabilidad
ConversationMemory	Wrapper de ConversationBufferWindowMemory con AF
build_memory(window_size)	Factory function — instancia la memoria con parámetro
format_history_prompt(text)	Serializa el historial para inyección en prompts
ContextualAgent	ManuelitaAgent + ConversationMemory con detección c
ContextualAgent.chat(question)	API pública del agente — devuelve answer + enriched -
ContextualAgent.reset()	Reinicia la conversación (borra historial)

Detección de preguntas de seguimiento

El método _enrich_question() detecta si una pregunta hace referencia implícita al contexto anterior y, en ese caso, inyecta el historial reciente como prefijo. Las señales de follow-up incluyen:

- Preguntas que inician con «¿y...», «¿también...», «¿además...»
- Palabras referenciales: «allí», «ahí», «eso», «ese», «esa»
- Preguntas cortas (menos de 5 palabras) que implican referencia previa
- Expresiones como «¿qué más...», «¿cómo lo...», «mencionaste»

Test	Resultado
Memoria básica (save/load/reset)	✓ 100%
Ventana deslizante (window=2)	✓ Turnos viejos descartados correctamente
Detección de follow-up	✓ ≥85% de precisión
Diálogo end-to-end (6 turnos)	✓ 100% routing
Historial persistente en sesión	✓ ConversationBufferWindowMemory

7. Bloque 5 — Interfaz Streamlit Chat

El Bloque 5 actualiza la interfaz Streamlit del Módulo 1 —una UI de preguntas y respuestas simple— a un chat conversacional completo con historial visible, burbujas de mensaje diferenciadas por rol, badges de fuente por mensaje y selector de proveedor en el sidebar.

Comparativa Módulo 1 vs Módulo 2

Característica	Módulo 1	Módulo 2
Motor	ManuelitaQASystem (RAG Simple)	ComplexAgent (HybridRouter + NLP)
Historial visible	Últimas 5 preguntas (expandible)	Burbujas de chat acumulativas
Indicador de fuente	No	Badge por mensaje: RAG / Estructurado
Memoria conversacional	No	ConversationBufferWindowMemory
Selector de proveedor	Ollama / Gemini	Local / Ollama / Gemini
Nueva conversación	No	Botón reset en sidebar
Métricas de sesión	No	Turnos / RAG / Estructurado / Conversacional
Tabs	Resumen + FAQ	Chat + Resumen + FAQ

8. Evaluación Comparativa entre Modos

La siguiente tabla resume el rendimiento del sistema completo (Módulo 2) para cada uno de los tres modos de proveedor disponibles:

Métrica	gemini	local	ollama
Routing (Bloque 3)	100%	100%	100%
Score RAG	~95% (estimado)	86%	~60%
Score Estructurado	100%	100%	100%
Tiempo RAG	~3s	~15s	~10s
Tiempo Estructurado	~0ms	~0ms	~0ms
Memoria follow-up	✓ Alta calidad	✓ Routing OK	✓ Routing OK
Costo	API gratuita (límites)	Gratis (local)	Gratis (local)
Requiere internet	Sí	Solo setup	No

Recomendación: usar modo **gemini** para la mejor calidad de respuestas narrativas y modo **local** como fallback cuando no hay conexión a internet o se ha agotado la cuota de la API. El routing funciona al 100% en todos los modos.

9. Estructura de Archivos del Módulo 2

Archivo	Bloque	Descripción
src/langchain_app/rag_engine.py	1	Motor RAG: carga, indexa, recupera co
data_processed/markdown/key_facts_manuelita.md	1	Corpus Q&A para alta similitud en retri
data/vectorstore/{proveedor}/	1	Índice ChromaDB persistente (no en g
data/structured/manuelita_datos.json	2	Base de datos JSON con datos exacto
src/langchain_app/tools/structured_tool.py	2	Motor de consulta + integración LangC
src/langchain_app/agent.py	3	ManuelitaAgent (HybridRouter) + build
src/langchain_app/memory.py	4	ConversationMemory + ContextualAge
app.py	5	Interfaz Streamlit chat conversacional
scripts/test_rag_bloque1.py	1	Suite de tests RAG (6 preguntas)
scripts/test_structured_tool.py	2	Suite de tests estructurado (10 pregun
scripts/test_agente_bloque3.py	3	Suite de tests router (10 preguntas)
scripts/test_memoria_bloque4.py	4	Suite de tests memoria (4 sub-tests)
scripts/test_modulo2_completo.py	6	Suite integrada completa de Módulos 2
MODULO2.md	Todos	Documentación técnica del módulo
PROVIDERS.md	1	Guía de los tres modos de proveedor

10. Conclusiones

- **Router híbrido sobre ReAct:** El HybridRouter alcanza 100% de precisión de routing con cualquier LLM, incluyendo modelos pequeños como llama3.2:3b. El agente ReAct, aunque conceptualmente más elegante, requiere modelos de mayor capacidad para funcionar de forma fiable en español.
- **Q&A; corpus como solución de retrieval:** La creación del archivo `key_facts_manuelita.md` en formato Q&A; fue la decisión técnica más impactante del Bloque 1: elevó el score de 53% a 86% sin cambiar el modelo de embeddings, aprovechando que la similitud coseno entre «¿Cuál es el NIT?» y su respuesta directa es casi perfecta.
- **Separación de responsabilidades:** Dividir el sistema en herramientas especializadas (RAG para narrativo, JSON para factual) elimina las alucinaciones en datos exactos y reduce la latencia de un 90% de las preguntas cotidianas a ~0ms.
- **Memoria como enhancer no como dependencia:** La memoria conversacional enriquece opcionalmente las preguntas de seguimiento pero no afecta al routing ni a las respuestas de datos estructurados, manteniendo la fiabilidad del sistema base.
- **Flexibilidad multi-proveedor:** La arquitectura soporta tres modos de operación (gemini/local/ollama) con la misma interfaz de usuario, permitiendo elegir entre calidad máxima (Gemini) y operación completamente offline (local/ollama).